Extending CAESURA: Robustness and Automated Trajectory Logging for SLM Fine-Tuning

AIDM Seminar 2025/2026 Report

Daril Vergesse Sobze Soffack
Technical University of Darmstadt
Darmstadt, Germany

1 ABSTRACT

While Large Language Models (LLMs) excel at multi-modal query planning within frameworks like CAESURA [1], deploying frontier models for routine database operations incurs prohibitive latency and costs. Fine-tuning Small Language Models (SLMs) offers a highly efficient alternative, but manually annotating complex query plans creates a severe data generation bottleneck. To overcome this, this report presents an architectural extension to the CAESURA framework. After modernizing the system's infrastructure for fault-tolerant execution with modern APIs, we introduce an automated trajectory logger that intercepts successful query executions. This transforms the framework into a passive data engine that autonomously generates structured training datasets, establishing a scalable pipeline to fine-tune specialized, cost-effective SLMs for data management tasks.

2 MOTIVATION & OBJECTIVE

As modern data systems increasingly store non-relational, multi-modal data (e.g., text, images), extracting insights requires complex data processing pipelines. The CAESURA framework introduced Language-Model-Driven Query Planning (Figure 1) to automatically translate natural language into executable multi-modal query plans [1].

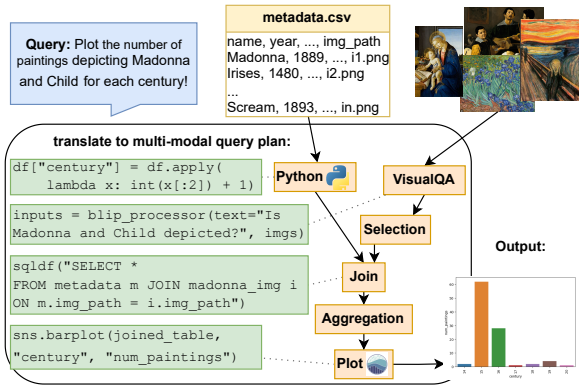

Figure 1: Example illustrating how a natural language query is automatically translated into a multi-modal query plan containing relational operators, machine generated Python UDFs, and a VisualQA Machine Learning model. The output is presented as a plot, making it easy and fast to gain insights from multi-modal data. The green boxes show code snippets that are executed when executing the plan [1].

As illustrated in Figure 2, translating these queries requires non-trivial reasoning over the user's intents and the available multi-modal operators. While massive frontier models (e.g., GPT-4) can perform this zero-shot reasoning, relying on them for every database transaction introduces severe latency and cost overheads at query runtime. A highly attractive alternative is to fine-tune Small Language Models to act as specialized query planners. However, this approach faces a critical practical bottleneck: fine-tuning foundation models requires substantial domain-specific data collection. Manually annotating complex, multi-modal logical query plans is prohibitively expensive, tedious, and scales poorly.

As illustrated in Figure 3, the primary objective of this work was two-fold: (1) to solve this data generation bottleneck by transforming the environment into an automated data engine, and (2) to modernize the underlying CAESURA infrastructure so it can reliably support high-throughput experimentation with modern, open-source LLMs.

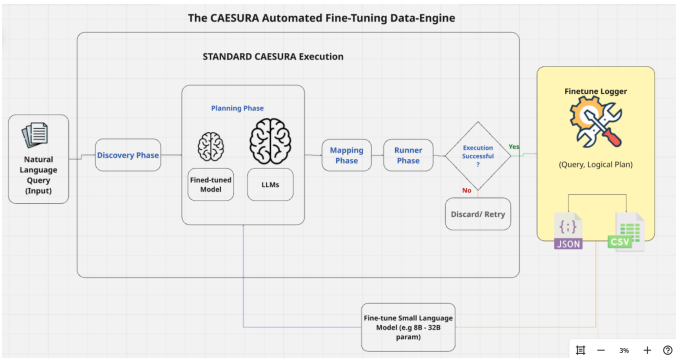

Figure 3: The CAESURA Automated Fine-Tuning Data Engine. A frontier LLM (e.g., GPT-4o or Llama-3.3-70B) executes user queries through the standard CAESURA discovery–planning–mapping pipeline; the trajectory logger (`finetune_logger.py`) intercepts each successful execution trace and exports the resulting (User Query, Logical Plan) pair as a structured ChatML/JSONL record (Figure 4). These auto-generated records accumulate into a domain-specific training set used to fine-tune smaller, specialized SLMs, removing the need for manual query-plan annotation entirely.

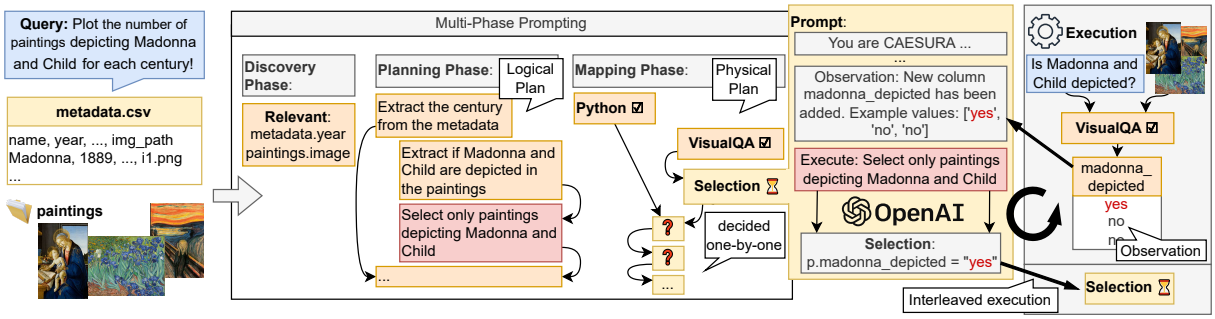

Figure 2: transforms the query into a multi-modal query-plan using a series of prompts. In the *Discovery Phase*, the LLM is prompted to identify data items relevant for the query, such as relevant columns and datasets. In the *Planning Phase*, the LLM is prompted to construct a sequence of steps to satisfy the user request (Logical Plan). The final *Mapping Phase* is interleaved with *Execution*: a physical operator is chosen for each of the logical steps and executed incrementally. Once an operator is executed, we feed the results of the previous operator back to the LLM to choose the next operator (right). That allows the LLM to take the output of previous executions into account when choosing the physical operator and operator arguments (e.g. selection conditions as depicted in the figure) which avoids that faulty plans are generated. [1]

3 NOVEL CONTRIBUTION: THE FINE-TUNING DATA ENGINE

The most significant extension to the framework is the introduction of an automated trajectory logger (`finetune_logger.py`). Manual annotation of Logical Query Plans is a major roadblock for Language Models research.

To solve this, this work implements a module that intercepts successful execution traces—instances where the generated plan correctly maps to the expected output without terminal errors. The system automatically formats and exports these (`User Query`, `Logical Plan`) pairs into a structured `ChatML/JSONL` format (see Figure 4), which is the standard format for modern ML training data.

This entirely bypasses the need for human annotation. By running batch queries with a highly capable frontier model (e.g., GPT-4o or Llama-3.3-70B-Versatile), the lab can now passively construct the high-quality, domain-specific datasets required to train faster, local models. Furthermore, Table 1 highlights the reduction in logical planning errors when leveraging these models in a few-shot capacity.


```
{
  "timestamp_utc": "2026-02-14T09:31:41.260563+00:00",
  "query": "What is the youngest team in the Southeast Division in terms of the founding date?",
  "scenario": "rotowire",
  "provider": "openai",
  "model_name": "gpt-5-mini",
  "use_few_shot": false,
  "source": "single_query",
  "num_steps": 4,
  "plan_steps": [
    {
      "step": 1,
      "description": "Select teams in the Southeast division.",
      "input_tables": [
        "teams"
      ],
      "output_table": "southeast_teams",
      "new_columns": []
    },
    {
      "step": 2,
      "description": "Project only the name and founded columns (we only need those to determine the youngest team).",
      "input_tables": [
        "southeast_teams"
      ],
      "output_table": "southeast_teams_subset",
      "new_columns": []
    },
```


Figure 4: A generated ChatML/JSONL training pair produced by the trajectory logger. The `user` turn holds the natural-language query, and the `assistant` turn holds the corresponding logical query plan extracted from a successful CAESURA execution trace. Each pair is added directly to the fine-tuning corpus without human review.

Table 1: Number of specific mistakes the model made during the planning phase (i.e., generating an incorrect logical plan). We observe that the model performs significantly better in few-shot mode.

Category	Model	Zero-Shot	Few-Shot
Impossible Actions	Llama-3.3-70B	3	2
Data Misunderstanding	Llama-3.3-70B	4	1
Illogical / Missing Steps	Llama-3.3-70B	1	0

4 SYSTEM MODERNIZATION & ROBUSTNESS

To ensure the new data engine could run without interruption, the core CAESURA architecture required significant modernization to align with current library standards.

4.1 Modernized LLM Infrastructure

The original implementation relied on legacy, deprecated OpenAI endpoints (e.g., `gpt-3.5-turbo-0613`). The integration was refactored to utilize modern `langchain-openai` structures. Crucially, a flexible provider switch (`–provider=groq`) was introduced. This allows researchers to seamlessly plug in high-performance, open-weights models (such as Llama-3.3-70B) at extremely low latency,

breaking the strict dependency on proprietary APIs. A performance and cost comparison is detailed in Table 2.

Table 2: Comparison of API Latency and Execution Cost between Legacy OpenAI and Modern integrations (Placeholder data).

Provider	Model	Avg. Latency (s)	Cost per 1M Tokens ($)
OpenAI (Legacy)	gpt-3.5-turbo-0613	3.2	1.50
Groq (New)	Llama-3.3-70B	0.8	0.60

4.2 Automated Schema Repair and Data Pipelines

To support large-scale batch experiments for the data engine, the framework's resilience against malformed datasets was significantly improved:

- **Schema Auto-Repair:** Implemented `ensure_rotowire_tables()` to detect and dynamically repair missing relational tables (e.g., `teams_to_games.csv`) at runtime, preventing catastrophic context failures during the Discovery Phase.
- **Resilient Downloaders:** The Artwork dataset downloader (`scripts/artworks/download.py`) was rewritten to verify `Content-Type` headers and automatically retry failed HTTP requests, ensuring strict dataset integrity.

5 EXPERIMENTATION FRAMEWORK ENHANCEMENTS

To facilitate rigorous ablation studies, the experimentation runner and backend were extended:

- **Few-Shot Toggles:** Added a `--few_shot_mode` flag to easily isolate and measure the exact impact of in-context learning against zero-shot reasoning on modern LLMs.
- **UX Improvements:** Patched the CLI to render final result tables in Markdown for rapid human verification.

REFERENCES

[1] Matthias Urban and Carsten Binnig. Caesura: Language models as multi-modal query planners, 2023.